EXPT.NO : 1A	ARRAY BASED IMPLEMENTATION OF STACK
DATE:	

AIM

ALGORITHM

PROGRAM

```
class Arraystack():
   def __init__(self):
      self.data=[]
   def __len__(self):
         return len(self.data)
   def is_empty(self):
      return len(self.data)==0
   def push(self,e):
      self.data.append(e)
   def top(self):
      if self.is_empty():
         raise Empty(Stack is empty)
      return self.data[-1]
   def pop(self):
       if self.is_empty():
         raise Empty(Stack is empty)
       return self.data.pop()

S=Arraystack()
S.push(90)
S.push(37)
print(len(S))
print(S.pop())
print(S.is_empty())
print(S.pop())
print(S.is_empty())
S.push(67)
S.push(39)
```

```
print(S.top())
S.push(45)
print(len(S))
print(S.pop())
S.push(67)
```

OUTPUT

```
>>>
============== RESTART: C:/Users/2025PECAI713.PEC/Documents/1st.py =========
2
37
False
90
True
39
3
45
>>>
```

RESULT

EXPT.NO : 1A	ARRAY BASED IMPLEMENTATION OF QUEUE
DATE:	

AIM

ALGORITHM

PROGRAM

```
class Arrayqueue:
    DEFAULT_CAPACITY=5
    def __init__(self):
        self.data=[None]
        self.size=0
        self.front=0
    def __len__(self):
        return self.size()
    def is_empty(self):
        return self.size==0
    def first(self):
        if self.is_empty():
            raise Exception("Queue is empty")
        return self.data[self.front]
    def display(self):
        for i in self.data:
            print(i)
    def dequeue(self):
        if self.is_empty():
            raise Exception("Queue is empty")
        answer=self.data[self.front]
        self.data[self.front]=None
        self.front=(self.front+1)%len(self.data)
        self.size-=1
        return answer
    def enqueue(self,e):
        if self.size==len(self.data):
```

```
        self.resize(2*len(self.data))
    avail=(self.front+self.size)%len(self.data)
    self.data[avail]=e
    self.size+=1
  def resize(self,cap):
    old=self.data
    self.data=[None]*cap
    walk=self.front
    for k in range(self.size):
        self.data[k]=old[walk]
        walk=(1+walk)%len(old)
    self.front=0
q=Arrayqueue()
q.enqueue(64)
q.enqueue(77)
q.enqueue(69)
print(q.dequeue())
print(q.is_empty())
print(q.first())
q.display()
```

OUTPUT

```
>>>
========== RESTART: C:\Users\2025PECAI713.PEC\Documents\untitled.py ========
1
False
2
None
2
3
None
None
<<<
```

RESULT

EXPT.NO : 2	EVALUATING POSTFIX EXPRESSION
DATE:	

AIM

ALGORITHM

PROGRAM

```
def evaluate_postfix(postfix_expr):
    stack = []
    operators = {'+', '-', '*', '/', '%', '^'}
    for elem in postfix_expr:
        if elem not in operators:
            stack.append(float(elem))
        else:
            b = stack.pop()
            a = stack.pop()
            if elem == '+':
                stack.append(a + b)
            elif elem == '-':
                stack.append(a - b)
            elif elem == '*':
                stack.append(a * b)
            elif elem == '/':
                stack.append(a / b)
            elif elem == '%':
                stack.append(a % b)
            elif elem == '^':
                stack.append(a ** b)
    return stack.pop()
postfix = "236*+"
print("{}: {}".format(postfix, evaluate_postfix(postfix)))
postfix = "23*6+"
print("{}: {}".format(postfix, evaluate_postfix(postfix)))
postfix = "23*42/+"
print("{}: {}".format(postfix, evaluate_postfix(postfix)))
```

OUTPUT

```
>>>
========== RESTART: C:\Users\2025PECAI713.PEC\Documents\untitled.py ====
236*+: 20.0
23*6+: 12.0
23*42/+: 8.0
```

RESULT

EXPT.NO : 3	BINARY SEARCH TREE OPERATIONS
DATE:	

AIM

ALGORITHM

PROGRAM

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
def inorder(root):
    if root is not None:
        inorder(root.left)
        print(root.key)
        inorder(root.right)
def insert(node, key):
    if node is None:
        return Node(key)
    if key < node.key:
        node.left = insert(node.left, key)
    else:
        node.right = insert(node.right, key)
    return node
def minValueNode(node):
    current = node
    while current.left is not None:
        current = current.left
    return current
def deleteNode(root, key):
    if root is None:
        return root
    if key < root.key:
        root.left = deleteNode(root.left, key)
```

```
    elif key > root.key:
        root.right = deleteNode(root.right, key)
    else:
        if root.left is None:
            temp = root.right
            root = None
            return temp
        elif root.right is None:
            temp = root.left
            root = None
            return temp
        temp = minValueNode(root.right)
        root.key = temp.key
        root.right = deleteNode(root.right, temp.key)
    return root
root = None
root = insert(root, 15)
root = insert(root, 10)
root = insert(root, 20)
root = insert(root, 8)
root = insert(root, 12)
root = insert(root, 16)
root = insert(root, -2)
print("Inorder traversal of the given tree")
inorder(root)
print("\nDelete 12")
root = deleteNode(root, 12)
print("Inorder traversal of the modified tree")
inorder(root)
```

OUTPUT

```
>>
========== RESTART: C:\Users\2025PECAI713.PEC\Documents\untitled.py ====
Inorder traversal of the given tree
-2
8
10
12
15
16
20

Delete 12
Inorder traversal of the modified tree
-2
8
10
15
16
20
>>
```

RESULT

EXPT.NO : 4	AVL TREE IMPLEMENTATION
DATE:	

AIM

ALGORITHM

PROGRAM

```
class treeNode(object):
    def __init__(self, value):
        self.value = value
        self.l = None
        self.r = None
        self.h = 1
class AVLTree(object):
    def insert(self, root, key):
        if not root:
            return treeNode(key)
        elif key < root.value:
            root.l = self.insert(root.l, key)
        else:
            root.r = self.insert(root.r, key)
        root.h = 1 + max(self.getHeight(root.l),
                         self.getHeight(root.r))
        b = self.getBal(root)
        if b > 1 and key < root.l.value:
            return self.rRotate(root)
        if b < -1 and key > root.r.value:
            return self.lRotate(root)
        if b > 1 and key > root.l.value:
            root.l = self.lRotate(root.l)
            return self.rRotate(root)
        if b < -1 and key < root.r.value:
            root.r = self.rRotate(root.r)
            return self.lRotate(root)
        return root
```

```
def lRotate(self, z):
    y = z.r
    T2 = y.l
    y.l = z
    z.r = T2
    z.h = 1 + max(self.getHeight(z.l),
                  self.getHeight(z.r))
    y.h = 1 + max(self.getHeight(y.l),
                  self.getHeight(y.r))
    return y
def rRotate(self, z):
    y = z.l
    T3 = y.r
    y.r = z
    z.l = T3
    z.h = 1 + max(self.getHeight(z.l),
                  self.getHeight(z.r))
    y.h = 1 + max(self.getHeight(y.l),
                  self.getHeight(y.r))
    return y
def getHeight(self, root):
    if not root:
        return 0
    return root.h
def getBal(self, root):
    if not root:
        return 0
    return self.getHeight(root.l) - self.getHeight(root.r)
def preOrder(self, root):
    if not root:
        return
```

```
        print("{0} ".format(root.value), end="")
        self.preOrder(root.l)
        self.preOrder(root.r)
Tree = AVLTree()
root = None
root = Tree.insert(root, 1)
root = Tree.insert(root, 2)
root = Tree.insert(root, 3)
root = Tree.insert(root, 4)
root = Tree.insert(root, 5)
root = Tree.insert(root, 6)
print("Preorder traversal of the constructed AVL tree is")
Tree.preOrder(root)
print()
```

OUTPUT

```
>>>
========== RESTART: C:\Users\2025PECAI713.PEC\Documents\untitled.py ===
Preorder traversal of the constructed AVL tree is
4 2 1 3 5 6
```

RESULT

EXPT.NO : 5	RED BLACK TREE IMPLRMRNTATION
DATE:	

AIM

ALGORITHM

PROGRAM

```
import sys
class Node():
    def __init__(self, item):
        self.item = item
        self.parent = None
        self.left = None
        self.right = None
        self.color = 1
class RedBlackTree():
    def __init__(self):
        self.TNULL = Node(0)
        self.TNULL.color = 0
        self.TNULL.left = None
        self.TNULL.right = None
        self.root = self.TNULL
    def pre_order_helper(self, node):
        if node != self.TNULL:
            sys.stdout.write(str(node.item) + " ")
            self.pre_order_helper(node.left)
            self.pre_order_helper(node.right)
    def in_order_helper(self, node):
        if node != self.TNULL:
            self.in_order_helper(node.left)
            sys.stdout.write(str(node.item) + " ")
            self.in_order_helper(node.right)
    def post_order_helper(self, node):
        if node != self.TNULL:
            self.post_order_helper(node.left)
            self.post_order_helper(node.right)
```

```
        sys.stdout.write(str(node.item) + " ")
    def search_tree_helper(self, node, key):
        if node == self.TNULL or key == node.item:
            return node
        if key < node.item:
            return self.search_tree_helper(node.left, key)
        return self.search_tree_helper(node.right, key)
    def minimum(self, node):
        while node.left != self.TNULL:
            node = node.left
        return node
    def left_rotate(self, x):
        y = x.right
        x.right = y.left
        if y.left != self.TNULL:
            y.left.parent = x
        y.parent = x.parent
        if x.parent == None:
            self.root = y
        elif x == x.parent.left:
            x.parent.left = y
        else:
            x.parent.right = y
        y.left = x
        x.parent = y
    def right_rotate(self, x):
        y = x.left
        x.left = y.right
        if y.right != self.TNULL:
            y.right.parent = x
        y.parent = x.parent
```

```
    if x.parent == None:
        self.root = y
    elif x == x.parent.right:
        x.parent.right = y
    else:
        x.parent.left = y
    y.right = x
    x.parent = y
def fix_insert(self, k):
    while k.parent.color == 1:
        if k.parent == k.parent.parent.right:
            u = k.parent.parent.left
            if u.color == 1:
                u.color = 0
                k.parent.color = 0
                k.parent.parent.color = 1
                k = k.parent.parent
            else:
                if k == k.parent.left:
                    k = k.parent
                    self.right_rotate(k)
                k.parent.color = 0
                k.parent.parent.color = 1
                self.left_rotate(k.parent.parent)
        else:
            u = k.parent.parent.right
            if u.color == 1:
                u.color = 0
                k.parent.color = 0
                k.parent.parent.color = 1
                k = k.parent.parent
```

```
            else:
                if k == k.parent.right:
                    k = k.parent
                    self.left_rotate(k)
                k.parent.color = 0
                k.parent.parent.color = 1
                self.right_rotate(k.parent.parent)
        if k == self.root:
            break
    self.root.color = 0
def insert(self, key):
    node = Node(key)
    node.parent = None
    node.item = key
    node.left = self.TNULL
    node.right = self.TNULL
    node.color = 1
    y = None
    x = self.root
    while x != self.TNULL:
        y = x
        if node.item < x.item:
            x = x.left
        else:
            x = x.right
    node.parent = y
    if y == None:
        self.root = node
    elif node.item < y.item:
        y.left = node
    else:
```

```
            y.right = node
        if node.parent == None:
            node.color = 0
            return
        if node.parent.parent == None:
            return
        self.fix_insert(node)
    def rb_transplant(self, u, v):
        if u.parent == None:
            self.root = v
        elif u == u.parent.left:
            u.parent.left = v
        else:
            u.parent.right = v
        v.parent = u.parent
    def delete_node_helper(self, node, key):
        z = self.TNULL
        while node != self.TNULL:
            if node.item == key:
                z = node
            if node.item <= key:
                node = node.right
            else:
                node = node.left
        if z == self.TNULL:
            print("Cannot find key in the tree")
            return
        y = z
        y_original_color = y.color
        if z.left == self.TNULL:
            x = z.right
```

```
        self.rb_transplant(z, z.right)
    elif z.right == self.TNULL:
        x = z.left
        self.rb_transplant(z, z.left)
    else:
        y = self.minimum(z.right)
        y_original_color = y.color
        x = y.right
        if y.parent == z:
            x.parent = y
        else:
            self.rb_transplant(y, y.right)
            y.right = z.right
            y.right.parent = y
        self.rb_transplant(z, y)
        y.left = z.left
        y.left.parent = y
        y.color = z.color
def delete_node(self, item):
    self.delete_node_helper(self.root, item)
def print_helper(self, node, indent, last):
    if node != self.TNULL:
        sys.stdout.write(indent)
        if last:
            sys.stdout.write("R----")
            indent += "     "
        else:
            sys.stdout.write("L----")
            indent += "|    "
        s_color = "RED" if node.color == 1 else "BLACK"
        print(str(node.item) + "(" + s_color + ")")
```

```
            self.print_helper(node.left, indent, False)
            self.print_helper(node.right, indent, True)
    def print_tree(self):
        self.print_helper(self.root, "", True)
if __name__ == "__main__":
    bst = RedBlackTree()
    bst.insert(55)
    bst.insert(40)
    bst.insert(65)
    bst.insert(60)
    bst.insert(75)
    bst.insert(57)
    bst.print_tree()
    print("\nAfter deleting an element")
    bst.delete_node(40)
    bst.print_tree()
```

OUTPUT

```
>>
======== RESTART: C:\Users\2025PECAI713.PEC\Desktop\DSAA\program\EXP5.py ======
R----55(BLACK)
     L----40(BLACK)
     R----65(RED)
          L----60(BLACK)
          |    L----57(RED)
          R----75(BLACK)

After deleting an element
R----55(BLACK)
     R----65(RED)
          L----60(BLACK)
          |    L----57(RED)
          R----75(BLACK)
>>
```

RESULT

EXPT.NO : 6	QUICK SORT
DATE:	

AIM

ALGORITHM

PROGRAM

```
def quickSort(arr):
    if len(arr) <= 1:
        return arr
    else:
        pivot = arr[0]
        left = []
        right = []
        for i in range(1, len(arr)):
            if arr[i] < pivot:
                left.append(arr[i])
            else:
                right.append(arr[i])
        return quickSort(left) + [pivot] + quickSort(right)
# Driver Code
arr = [10, 7, 8, 9, 1, 5]
print("Original array is:")
print(arr)
sorted_arr = quickSort(arr)
print("Sorted array is:")
print(sorted_arr)
```

OUTPUT

```
>>>
======== RESTART: C:\Users\2025PECAI713.PEC\Desktop\DSAA\program\EXP6.py ======
Original array is:
[10, 7, 8, 9, 1, 5]
Sorted array is:
[1, 5, 7, 8, 9, 10]
>>>
```

RESULT

EXPT.NO : 7	MINIMUM COST SPANNING TREE USING GREEDY APPROACH
DATE:	

AIM

ALGORITHM

PROGRAM

```
INF = 999999
N = 5   # Number of vertices in graph
# Creating graph by adjacency matrix method
G = [
   [0, 9, 75, 0, 0],
   [9, 0, 95, 19, 42],
   [75, 95, 0, 51, 66],
   [0, 19, 51, 0, 31],
   [0, 42, 66, 31, 0]
]
selected = [0, 0, 0, 0, 0]
no_edge = 0
selected[0] = True
print("Edge : Weight")
while (no_edge < N - 1):
   minimum = INF
   a = 0
   b = 0
   for m in range(N):
      if selected[m]:
         for n in range(N):
            if ((not selected[n]) and G[m][n]):
               if minimum > G[m][n]:
                  minimum = G[m][n]
                  a = m
                  b = n
   print(str(a) + "-" + str(b) + ":" + str(G[a][b]))
   selected[b] = True
```

```
no_edge += 1
```

OUTPUT

```
>>>
======== RESTART: C:\Users\2025PECAI713.PEC\Desktop\DSAA\program\EXP7.py ======
Edge : Weight
0-1:9
1-3:19
3-4:31
3-2:51
>>>
```

RESULT

EXPT.NO : 8	HUFFMAN CODING
DATE:	

AIM

ALGORITHM

PROGRAM

```
from collections import Counter
import heapq
class node:
    def __init__(self, freq, symbol, left=None, right=None):
        self.freq = freq
        self.symbol = symbol
        self.left = left
        self.right = right
        self.huff = ''
    def __lt__(self, nxt):
        return self.freq < nxt.freq
def printNodes(node, val=''):
    newVal = val + node.huff
    if node.left:
        printNodes(node.left, newVal)
    if node.right:
        printNodes(node.right, newVal)
    if not node.left and not node.right:
        print(node.symbol, ":", newVal)
# Main Program
chars = ['A', 'B', 'C', 'D']
freq = [5, 1, 6, 3]
nodes = []
for x in range(len(chars)):
    heapq.heappush(nodes, node(freq[x], chars[x]))
while len(nodes) > 1:
    left = heapq.heappop(nodes)
    right = heapq.heappop(nodes)
    left.huff = '0'
```

```
    right.huff = '1'
    newNode = node(left.freq + right.freq,
                   left.symbol + right.symbol,
                   left, right)
    heapq.heappush(nodes, newNode)
printNodes(nodes[0])
```

OUTPUT

```
>>
    ======== RESTART: C:\Users\2025PECAI713.PEC\Desktop\DSAA\program\EXP8.py ===
    C : 0
    B : 100
    D : 101
    A : 11
>>
```

RESULT

EXPT.NO : 9	SPARSE MATRIX TRANSPOSE
DATE:	

AIM

ALGORITHM

PROGRAM

```
sparseMatrix = [
    [0, 0, 3, 0, 4],
    [0, 0, 5, 7, 0],
    [0, 0, 0, 0, 0],
    [0, 2, 6, 0, 0]
]
size = 0
for i in range(4):
    for j in range(5):
        if sparseMatrix[i][j] != 0:
            size += 1
rows, cols = (3, size)
compactMatrix = [[0 for i in range(cols)] for j in range(rows)]
k = 0
for i in range(4):
    for j in range(5):
        if sparseMatrix[i][j] != 0:
            compactMatrix[0][k] = i
            compactMatrix[1][k] = j
            compactMatrix[2][k] = sparseMatrix[i][j]
            k += 1
for i in compactMatrix:
    print(i)
```

OUTPUT

```
>>>
    ======== RESTART: C:\Users\2025PECAI713.PEC\Desktop\DSAA\program\EXP9.py ====
    [0, 0, 1, 1, 3, 3]
    [2, 4, 2, 3, 1, 2]
    [3, 4, 5, 7, 2, 6]
>>>
```

RESULT

EXPT.NO : 10	ALL PAIRS SHORTEST PATH PROBLEM USING DYNAMIC PROGRAMMING
DATE:	

AIM

ALGORITHM

PROGRAM

```
INF = 99999
def floydWarshall(graph, n):
    dist = graph
    for k in range(n):
        for i in range(n):
            for j in range(n):
                dist[i][j] = min(dist[i][j],
                                 dist[i][k] + dist[k][j])
    print(dist)
    return dist
graph = [
    [0, 5, INF, INF],
    [50, 0, 15, 5],
    [30, INF, 0, 15],
    [15, INF, 5, 0] ]
floydWarshall(graph, 4)
```

OUTPUT

```
======= RESTART: C:\Users\2025PECAI713.PEC\Desktop\DSAA\program\EXP10.py ====
[[0, 5, 15, 10], [20, 0, 10, 5], [30, 35, 0, 15], [15, 20, 5, 0]]
>>>
```

RESULT

EXPT.NO : 11	BACKTRACKING – N QUEENS PROBLEM
DATE:	

AIM

ALGORITHM

PROGRAM

```
N = 4
def printSolution(board):
    for i in range(N):
        for j in range(N):
            print(board[i][j], end=" ")
        print()
def isSafe(board, row, col):
    for i in range(col):
        if board[row][i] == 1:
            return False
    i = row
    j = col
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1
    i = row
    j = col
    while i < N and j >= 0:
        if board[i][j] == 1:
            return False
        i += 1
        j -= 1
    return True
def solveNQUtil(board, col):
    if col >= N:
        return True
    for i in range(N):
```

```
        if isSafe(board, i, col):
            board[i][col] = 1
            if solveNQUtil(board, col + 1):
                return True
            board[i][col] = 0
    return False
def solveNQ():
    board = [[0 for x in range(N)] for y in range(N)]
    if not solveNQUtil(board, 0):
        print("Solution does not exist")
        return False
    printSolution(board)
    return True
# Driver Code
solveNQ()
```

OUTPUT

```
>>>
======= RESTART: C:\Users\2025PECAI713.PEC\Desktop\DSAA\program\EXP11.py ====
0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0
>>>
```

RESULT

EXPT.NO : 12	PROGRAM TO IMPLEMENT TRAVELLING SALESMAN PROBLEM
DATE:	

AIM

ALGORITHM

PROGRAM

```
import math
maxsize = float('inf')
def copyToFinal(curr_path):
    final_path[:N + 1] = curr_path[:]
    final_path[N] = curr_path[0]
def firstMin(adj, i):
    min = maxsize
    for k in range(N):
        if adj[i][k] < min and i != k:
            min = adj[i][k]
    return min
def secondMin(adj, i):
    first, second = maxsize, maxsize
    for j in range(N):
        if i == j:
            continue
        if adj[i][j] <= first:
            second = first
            first = adj[i][j]
        elif adj[i][j] <= second and \
                adj[i][j] != first:
            second = adj[i][j]
    return second
def TSPRec(adj, curr_bound, curr_weight,
            level, curr_path, visited):
    global final_res
    if level == N:
        if adj[curr_path[level - 1]][curr_path[0]] != 0:
            curr_res = curr_weight + adj[curr_path[level - 1]][curr_path[0]]
```

```
            if curr_res < final_res:
                copyToFinal(curr_path)
                final_res = curr_res
        return
    for i in range(N):
        if adj[curr_path[level - 1]][i] != 0 and \
                visited[i] == False:
            temp = curr_bound
            curr_weight += adj[curr_path[level - 1]][i]
            if level == 1:
                curr_bound -= ((firstMin(adj, curr_path[level - 1]) +
                                firstMin(adj, i)) / 2)
            else:
                curr_bound -= ((secondMin(adj, curr_path[level - 1]) +
                                firstMin(adj, i)) / 2)
            if curr_bound + curr_weight < final_res:
                curr_path[level] = i
                visited[i] = True
                TSPRec(adj, curr_bound, curr_weight,
                       level + 1, curr_path, visited)
            curr_weight -= adj[curr_path[level - 1]][i]
            curr_bound = temp
            visited = [False] * len(visited)
            for j in range(level):
                if curr_path[j] != -1:
                    visited[curr_path[j]] = True
def TSP(adj):
    curr_bound = 0
    curr_path = [-1] * (N + 1)
    visited = [False] * N
    for i in range(N):
```

```
        curr_bound += (firstMin(adj, i) +
                       secondMin(adj, i))
    curr_bound = math.ceil(curr_bound / 2)
    visited[0] = True
    curr_path[0] = 0
    TSPRec(adj, curr_bound, 0, 1, curr_path, visited)
adj = [[0, 10, 15, 20],
       [10, 0, 35, 25],
       [15, 35, 0, 30],
       [20, 25, 30, 0]]
N = 4
final_path = [None] * (N + 1)
visited = [False] * N
final_res = maxsize
TSP(adj)
print("Minimum cost :", final_res)
print("Path Taken : ", end=' ')
for i in range(N + 1):
    print(final_path[i], end=' ')
```

OUTPUT

```
>>>
    ======= RESTART: C:\Users\2025PECAI713.PEC\Desktop\DSAA\program\EXP12.py =====
    Minimum cost : 80
    Path Taken :  0 1 3 2 0
>>>
```

RESULT

EXPT.NO : 13	APPROXIMATION ALGORITHM FOR KNAPSACK PROBLEM
DATE:	

AIM

ALGORITHM

PROGRAM

```
class Item:
    def __init__(self, value, weight):
        self.value = value
        self.weight = weight
class Node:
    def __init__(self, level, profit, bound, weight):
        self.level = level
        self.profit = profit
        self.bound = bound
        self.weight = weight
def bound(u, n, W, arr):
    if u.weight >= W:
        return 0
    profit_bound = u.profit
    j = u.level + 1
    totweight = u.weight
    while j < n and totweight + arr[j].weight <= W:
        totweight += arr[j].weight
        profit_bound += arr[j].value
        j += 1
    if j < n:
        profit_bound += (W - totweight) * arr[j].value / arr[j].weight
    return profit_bound
def knapsack(W, arr, n):
    arr.sort(key=lambda x: x.value / x.weight, reverse=True)
    queue = []
    u = Node(-1, 0, 0, 0)
    v = Node(-1, 0, 0, 0)
    u.bound = bound(u, n, W, arr)
```

```
    queue.append(u)
    max_profit = 0
    while queue:
        u = queue.pop(0)
        if u.level == -1:
            v.level = 0
        if u.level == n - 1:
            continue
        v.level = u.level + 1
        v.weight = u.weight + arr[v.level].weight
        v.profit = u.profit + arr[v.level].value
        if v.weight <= W and v.profit > max_profit:
            max_profit = v.profit
        v.bound = bound(v, n, W, arr)
        if v.bound > max_profit:
            queue.append(Node(v.level, v.profit, v.bound, v.weight))
        v.weight = u.weight
        v.profit = u.profit
        v.bound = bound(v, n, W, arr)
        if v.bound > max_profit:
            queue.append(Node(v.level, v.profit, v.bound, v.weight))
    return max_profit
W = 10
arr = [Item(40, 2), Item(50, 3.14), Item(100, 1.98), Item(95, 5), Item(30, 3)]
n = len(arr)
print("Maximum profit is", knapsack(W, arr, n))
```

OUTPUT

```
>>>
======= RESTART: C:\Users\2025PECAI713.PEC\Desktop\DSAA\program\EXP13.py ====
Maximum profit is 235
>>>
```

RESULT